Principal AI Engineer — Technical Task

Effort: 2 days

Deliverable: private Git repo

How we want you to work

This is a vibe-coding task. You're expected to build it with AI coding assistants (Claude Code, Cursor, Copilot, Aider — your choice) doing most of the typing. We're not testing whether you can write a tokenizer from scratch; we're testing whether you can **direct an AI agent to ship a thoughtful Arabic-aware system**, catch its mistakes, and explain your decisions.

Be open about it. The AI journal (below) is graded.

The scenario

Build **Murshid** — an Arabic-first assistant that answers GCC citizens' questions about government services (iqama, traffic fines, sponsorship transfer, municipal permits) in their own dialect, grounded in official source documents.

Starter data in /data:

- ~20 fabricated Arabic source documents (FAQ-style).
- ~15 user questions: MSA, Saudi dialect, Khaleeji, some code-switched ("متى يخلص my iqama؟").
- ~8 gold answers for evaluation.

Runs locally in under 10 minutes for a reviewer.

What to build

A working pipeline that:

1. **Ingests and retrieves** from the Arabic sources. You must consciously handle Arabic normalization (alef forms, ya/alef-maqsura, ta-marbuta, diacritics) and pick an embedding model that's actually good at Arabic.
2. **Detects the user's register** (MSA vs dialect) and answers in the same register, with citations.
3. **Escalates or refuses** when grounding is weak — never fabricates a government policy.

4. **Has at least one eval** you can run and explain.

Model choice is yours — GPT, Claude, Gemini, or an Arabic-native model (ALLaM, Jais 2, Fanar, AceGPT, Hala, Falcon-Arabic, SILMA). **Defend your choice in the ADR.**

Deliverables

File	What goes in it
README.md	One-command setup + demo. Tell the reviewer which 3 questions to try (one MSA, one dialect, one that should escalate).
docs/ARCHITECTURE.md	System diagram (any format), component contracts, **3 ADRs** — one of them must be your Arabic embedding/model choice. One short section on Arabic-specific risks (diglossia, code-switching, hallucinated policy, RTL bugs). One paragraph on GCC production gaps (PDPL, data residency, Arabic agent escalation).
docs/AI_JOURNAL.md	Which AI tools you used. **3 prompts** worth showing — what you asked, what you accepted, what you rejected. **One AI mistake you caught** (bonus: Arabic-specific). **One thing you let the agent do autonomously** and the guardrails you set. Honest reflection on where AI helped vs hurt.
docs/CREATIVE.md (1 page)	One unprompted thing you built. Suggestions: dialect classifier, dialect→MSA normalizer for retrieval, Arabic critic agent, MCP server with Arabic tool descriptions, Hijri-date module, Arabic red-team harness. Or your own idea.
Code	The thing itself.

How we evaluate (in order of importance)

1. **Arabic technical depth.** You treat Arabic as first-class, not a translation overlay. Your ADR shows you know the 2026 Arabic-model landscape.
2. **Vibe-coding fluency.** You moved fast *and* understand every line that ships. Journal proves it.
3. **Architecture & docs.** A reader can predict how the system behaves on a Khaleeji code-switched question without reading code.
4. **Trust thinking.** Hallucinated government policy is unacceptable. Show how you prevented it.
5. **Creativity.** The extension should be something an English-only engineer wouldn't have built.
6. **Engineering rigor.** Eval exists, errors handled, secrets out of repo, costs bounded.

We do **not** evaluate: feature count, UI polish, whether you're a native Arabic speaker, whether you used "our" stack.

Ground rules

- AI use is **expected**. Hidden AI use is a red flag; documented AI use is the point.
- No human collaborators or Arabic-speaking friends. Ask the AI; tell us in the journal.
- Time-box honestly. Stopping at 6 hours done > grinding 14 hours unfinished.
- Unfinished is fine if documented ("I'd do X next, here's why I didn't").
- Make calls on ambiguity, document them. That ambiguity is the task.

Submit

Email the repo link + a ≤150-word note: the decision you're most proud of, the one you'd revisit, and your LLM provider.